Modelling decisions: v14.0

> **Superseded by v16.0.** The shipped model is now `model/UFCEP6-0-3_Hospital_Patient_Pathway_v16.bpmn` : one hospital pool, three responsibility lanes holding nine team lanes, and the outside participants as black-box pools. Everything below describes the v14.0 fourteen-pool collaboration and is kept as the record of the release that was deployed and reviewed. What the shipped model contains is in 17-single-pool-operational-model.md.

> **Citations to files that are not in this tree.** An audit found 39 paths cited across 19 documents that no longer exist — the fourteen-pool model, the v1/v2 generators and the drawing scripts that went with them, and the Sprint 0 Node scaffold. Nothing was lost: all of it is in git history, and every citation is listed with what replaced it in 18-cited-files-no-longer-present.md.

Written to be read out. Each section records the decision, the reason for it, and the alternative that was set aside.

---

1. Why one diagram with fifteen pools, and why none of them is a black box

The case study names eleven teams and four outside organisations, plus the patient and the referrer. Since the brief allows one file and one diagram, the collaboration is drawn as fifteen pools stacked vertically. From v14 onwards all fifteen of them carry a process: the nine the hospital runs, and one for each of the six outside participants.

What matters is how much of each process gets drawn. A hospital pool is drawn in full, because the hospital controls the work and the sequence is what the audit trail has to show. An outside participant is drawn only as far as the boundary: the message that arrives, the one step that does the work, the answer that goes back. The case study describes nothing beyond that point, only what we send, what comes back, and what happens when nothing comes back. Anything more would be inventing a process nobody in the room can verify.

That is still better than a black box, and v1.0–v7.0 showed why. A black-box pool answers "who else is involved" and nothing else. Three of the six (the payment provider, the treatment and diagnostic services and the scheduling service) were reached only from service tasks inside the hospital pools. When those documentary dashed lines were thinned away in the readability release, the three pools were left with no connection at all: a box on the drawing that nothing pointed at. That reads as a modelling mistake rather than a decision.

v14 draws the outside side of each exchange and connects it:

outside participant	what its process shows	its step runs
Referring organisation	the referral going out	*(events only; the pack is prepared outside this system)*
Patient or representative	attending the appointment	*(events only)*

outside participant	what its process shows	its step runs
External Scheduling Service	search the diary, return slots	`scheduling.find-appointment-slots`
External Correspondence Service	print and dispatch the letter	`correspondence.dispatch-letter`
External Payment Service Provider	process the card transaction, return status and reference	`payment.process-transaction`
External Treatment, Laboratory and Imaging Services	check treatment and diagnostic capacity	`external-resources.check-availability`

Two decisions keep this cheap and honest. First, each supplier's step uses the same job type the hospital's own service task already calls, so the Java workers that already implement the four suppliers drive both ends: no new worker, no new job type, no new error code. Second, the hospital side was left alone. Its service tasks and error boundary events stay exactly as they were, which means a supplier that cannot answer is still `SCHEDULING_SERVICE_UNAVAILABLE`, `CORRESPONDENCE_SERVICE_FAILED`, `PAYMENT_PROVIDER_UNAVAILABLE` or `EXTERNAL_RESOURCE_UNAVAILABLE` on the hospital's own task. The two sides describe one exchange from opposite ends of the same job type.

The alternative, converting the twelve hospital-side supplier calls into message throw/catch pairs and deleting the eight error boundary events, would be a more orthodox BPMN collaboration. It was considered and rejected. It rewrites the exception architecture of a model that runs clean, invalidates the recorded runs in `docs/03-test-record.md`, and cannot be verified on a machine with no engine and no Maven to rebuild the workers. The proposal is written up as the next step in `docs/11-white-box-external-participants.md` §5.

2. Why messages rather than one long process

Every hand-off crosses a pool as a message flow, and each flow is backed by a real throw and catch pair. The collaboration therefore runs; it is not merely drawn.

The decision that needs explaining is the **correlation key**. Camunda 8 allows only one active instance per process definition per correlation key. A plain patient reference would mean the Consultant pool could exist only once at a time for a given patient, so a treatment review could not be in flight while a letter was being drafted. Every hand-off therefore appends a short purpose tag: `patientRef + "-referral-review"`, `+ "-funding"`, `+ "-refund"`.

That choice buys something for free. A second identical hand-off for the same patient and the same purpose is simply not correlated again, so nobody receives two appointment letters, two funding decisions or two refunds. Duplicate suppression lives in the key rather than in a separate check.

3. Gateway choices

Exclusive (34). Every decision point. Each one has exactly one default flow, with FEEL conditions on the rest, so an unexpected value fails safe into a defined path instead of stalling the instance. Three are worth naming:

- *Clinical decision*: accept, reject, redirect, or ask for more clinical detail. "More detail" is the default, because an unrecorded decision should come back to the clinician rather than be treated as a rejection.

- *Who is paying*: patient pays, or hospital / insurer / exemption. The funded route is the default, since most patients in the case study are never asked to pay, so the common path is the one that needs no condition.
- *What did the provider report*: approved, duplicate, or (default) declined or cancelled. Anything the provider says that we do not recognise is treated as "not paid", which is the safe reading.

Inclusive (3). Used where more than one branch can genuinely be live.

- *Patient notification* on Outpatient Bookings: the letter always goes out, and a telephone call is added only when the appointment is inside two weeks. The case study is explicit about that rule, and an exclusive gateway would force a choice between two things that both have to happen.
- Two joins to match those splits.

Parallel (2). Treatment confirmation. Once a schedule is confirmed, the patient is told and the Consultant is asked for the clinic letter. Both, always. A pair of parallel gateways says exactly that; an exclusive one would be a lie.

Event-based (2). Waiting for the funding decision, and waiting for the missing referral information. In both cases either a message arrives or a timer fires, whichever happens first, and the model cannot know which. A catch event plus a boundary timer would be two subscriptions on one activity; the event-based gateway makes the race explicit.

Exclusive merge vs parallel join. Wherever several branches reconverge on a task, they reconverge implicitly: that is an XOR merge, and it does not wait. The two parallel gateways in Treatment Confirmation are the only AND joins, and the only places where every token genuinely has to arrive.

4. Exception branches: what each one is for

The v1.0 audit found that most exception branches ended in a bare end event. A branch that stops is detection, not handling. Each one now has an owner and a way out.

Branch	Trigger	What happens	Why
Unreadable or incomplete pack	`REFERRAL_PACK_UNREADABLE`, `REFERRAL_PACK_INCOMPLETE`	Joins the missing-information loop: request from the referrer, wait 14 days, then close or re-check	A corrupt scan and a missing document need the same action: ask the referrer. Two separate dead ends would have been two ways to lose a referral
Scheduling service unavailable	`SCHEDULING_SERVICE_UNAVAILABLE`	A person decides: widen the search, offer the next available date, or escalate to the pathway team	The system must not silently book outside the period the Consultant asked for
Correspondence rejected	`CORRESPONDENCE_SERVICE_FAILED`	Pick another channel and retry	The letter still has to go out
Payment provider unavailable	`PAYMENT_PROVIDER_UNAVAILABLE`	Leave with the Finance Team, booking unconfirmed	Transient by nature; retries handle the first three attempts
Payment taken, no confirmation	`PAYMENT_CONFIRMATION_LOST`	Mark for investigation and ask a clinician about proceeding	The case study is specific: the money may have gone, so the transaction is investigated rather than charged again, but care must not wait for the paperwork

Branch	Trigger	What happens	Why
Duplicate charge reported	`paymentStatus = DUPLICATE`	Deduplicate before anything else	Money never leaves the account twice
External resource unavailable after three attempts	counted retry, cap 3	Release the provisional series (compensation) and hand the case to the pathway team, then retry the whole booking in three days	Keeps "still unavailable" from becoming nobody's job
Funding not approved in five days	timer	Chase, then flag the delay to the pathway team	Funding delays were previously invisible
Letter delayed past 7 days / 1 month / 3 months	three bands	Reminder, then Administrative Manager, then higher management	The three thresholds are given in the case study
Urgent clinical enquiry not picked up within an hour	non-interrupting timer	Escalate	The case study says urgency rules are not agreed, so the rule sits visibly in the model instead of being buried in code; it can be argued about

5. Compensation

Two things in this pathway commit a resource and can later have to be undone: the provisional appointment series, and a pencilled-in chemotherapy cycle slot.

Both get a compensation boundary event, a handler marked `isForCompensation`, and a compensation throw on the path that gives up. The handlers are typed `treatment.release-series` and `treatment.release-cycle-booking`.

One case is deliberately left out. Cancelling a paid appointment happens in a different process instance from the booking, and compensation does not cross instances in Camunda 8. That case goes through an explicit hand-off to the Finance Team's refund process instead. Using compensation there would have looked sophisticated and quietly not worked.

6. What was assumed

The case study leaves gaps. These are the calls made, and the reasoning.

1. **Urgency rules for clinical enquiries are not agreed.** The call handler sets the priority and a one-hour non-interrupting timer escalates anything not picked up. Recorded as an assumption to raise with the stakeholder group, not presented as a requirement.
2. **"Higher management" is not a named team.** The three-month escalation lands with the Administrative Management Team, whose pool documents that it goes to the executive team.
3. The Patient and the referring organisation are outside triggers. Their messages arrive from outside the modelled system, and they are published rather than thrown from a modelled task.
4. **Clinic letters are triggered when treatment is confirmed.** The seven-day target runs from the appointment, but the pathway only has a recorded event to hang the letter on at that point.

5. Financial figures are illustrative. Tariffs, funding limits and refund amounts come from the workers' own rules, not a real tariff.
6. **The Clinical Nurse Specialist administrative support team** named in the case study is folded into the nurse specialist pool rather than given its own lane, because the case study does not say which tasks are theirs.

7. Known limitations

Say these before anyone else does.

1. **Crossings.** 148 sequence-flow crossings against v1.0's 42. Spreading routes across corridors to remove overlaps means more of them cross. Zero overlaps and zero crossings are in tension; this model chose zero overlaps deliberately.
2. Four flows clip a boundary event by a few pixels. Cosmetic, and recorded rather than fixed because moving the boundary events further would detach them from the edge of the activity they belong to.
3. The canvas is a single very large diagram: 8698 × 16018. It is exported as SVG plus fifteen per-pool PNGs for that reason.
4. **The four external suppliers are stubs.** They are deterministic enough to demo and can be forced to fail, but there is no network call and no real provider. Swapping one for a real API means replacing one worker class.
5. Idempotency state lives in memory in a single JVM. A real system would use database constraints on the booking and payment references.
6. **The deployment batch ceiling (measured, not assumed).** Camunda 8 stores the whole BPMN resource inside every process definition record, so fifteen processes are fifteen copies of a 405 KB file. Against c8run 8.10.0-alpha5 the deployment is rejected:

`Can't append entry: ... valueType=PROCESS ... with size: 405002 this would exceed the maximum batch size. [ currentBatchEntryCount: 24, currentBatchSize: 4095764 ]`

Nine process records fit; the tenth does not. v7.0 stayed inside the limit at nine processes, which is why the supplier pools were left as black boxes. v14 draws all fifteen and ships with `EXTERNAL_PROCESSES_EXECUTABLE = False` in `tools/spec_v2.py`, so the six outside processes are documentation rather than deployable definitions, and the main file deploys its nine. The six go out separately from `tools/build_external_participants.py`, which produces a 30 KB file. Both deployments are verified. Raising `zeebe.broker.network.maxMessageSize` above 4 MB is the alternative; the switch is the one-line change back.

1. Three branches have not been run end to end: the declined-payment and confirmation-lost branches, and the three-month letter escalation. They are modelled and the workers support them, but they have not been demonstrated.
2. **The model has not been opened in Camunda Modeler** since the v2.0 changes. It deploys through the API and renders through bpmn-js, but the Modeler's own problem count has not been re-checked.